Cryptography Stack Exchange is a question and answer site for software developers, mathematicians and others interested in cryptography. It only takes a minute to sign up.

Sign up to join this community

Anybody can ask a question

Anybody can answer

The best answers are voted up and rise to the top

CRYPTOGRAPHY

What are the odds of collisions for a hash function with 256-bit output?

Asked 5 years, 1 month ago Active 3 months ago Viewed 15k times

23

12

There are some related questions on the net but I did not understand their solutions.

I am reading in a textbook about methods of finding a collision. It states to consider a collision for a hash function with a 256-bit output size and writes if we pick random inputs and compute the hash values, that we'll find a collision with high probability and if we choose just 2^{130} + 1 inputs, it turns out that there is a 99.8% chance at least two inputs will collide. It also says we can find a collision by looking at roughly the square root of the number of possible output results.

Questions:

1. What is the formula used to calculate that if we choose 2^{130} + 1 input at least 2 inputs will collide with a 99.8% probability?

   From my research it looks like this is related to the "birthday attack" problem, where you calculate first the probability that the hash inputs DO NOT collide and subtract this off from 1. Whenever I tried to plug and chug using the formulas I found online I wasn't getting the results the book stated.

   Edit: I tried using the formula P(A) = 1 - $\frac{k!}{(k-n)!k^n}$ and obtained 99.99...%. I do not know how the authors obtained the percentage 99.8%. Can someone please state the explicit equation used, so I can see how the book gets the 99.8% probability of collisions with 2^{130} + 1 randomly chosen inputs? Where does the +1 come from?

2. How is the square root function related to this? If I take square root of 256, I get 16. What does this mean in the context of collisions?
3. The size of the output is 256 bits. Does this mean the total number of possible outputs is 2^{256}?
4. Does the pigeonhole principle apply to this? You are taking an infinite amount of inputs and mapping them to a finite amount of outputs.

hash collision-resistance

Share Improve this question Follow

edited Aug 29 '16 at 20:37

asked Aug 27 '16 at 16:54
Max
377 1 4 11

That's elementary variation on the birthday problem. Yes b bits of output implies 2^b possible outputs (and you are not applying that, hence at least some of your problems). 2^b is what you should take the square root of to approximate the collision/birthday bound. The pigeonhole principle applies when there are more pigeon/hashes than there are holes/possible output values, thus in the context for immensely more hashes than for the collision bound. – fgrieu ♦ Aug 27 '16 at 17:40

Thanks for the response. Can you explain the formula used to calculate that with $2^{130}+1$ random inputs , it gives a probability of 99.8% that a collision will occur? I tried using the formula P(A) = 1 - $\frac{k!}{(k-n)!k^n}$ = 1 - $\frac{256!}{(256-130)!256^{130}}$ = 99.99...% and this not 99.8% from the book. This comes from: cs.duke.edu/courses/cps102/spring09/Lectures/L-18.pdf page 49. Can you explain the formula and give an example of how this calculation was done? – Max Aug 27 '16 at 17:55

My guess: The book used an approximation of the formula yielding "slightly" less precise answers. Also I *think* you need not to use bitlengths but rather actual values in your calculations, eg use 2^{256} instead if 256 (same for 130). – SEJPM ♦ Aug 27 '16 at 18:12

fgrieu, the book doesn't make it clear to me how many values/hashes you have. I assume 2^{256} possible output hashes and 2^{130} randomly chosen input hashes (from a possible infinite amount). The problem in question is related to: i.imgur.com/WcEk8xh.jpg. Do you know where the +1 is coming from? – Max Aug 27 '16 at 18:19

@max: you have the counts right (perhaps within 1 for hashes, ignore that for now and just assume $2^{130}+1$ means just above 2^{130}). But you have not quite plugged that right into the formula. And the numbers are so large that a calculator, or Excel, can't handle them. Your $3.4 \cdot 10^{38}$ is right, and gives you about the number of hashes for 50% odds of collisions. 2^{130} is more, and gives a larger probability of collision; all aligns well. – fgrieu ♦ Aug 27 '16 at 18:26

1 Answer

Active Oldest Votes

33

Birthday problem for cryptographic hashing, 101.

Let p_n be the probability of collision for a number n of random distinct inputs hashed to k possible values (that is, probability that at least two hashes are identical), on the assumption that the hash is perfect. That p_n is also the minimum probability of collision with no hypothesis

that the hash is perfect. That p_n is also the minimum probability of collision with no hypothesis on the hash.

Obviously, $p_0 = p_1 = 0$ (until there are at least two hashes, no collision is possible); and $p_j = 1$ if $j > k$ (by the pigeonhole principle, with more than k pigeons for k holes, at least one pigeon needs to share a hole with another).

Let $q_j = 1 - p_j$ be the probability of no collision for j hashes, with $q_0 = 1$. When we add another hash after there was no collision among the previous j hashes, the new hash has probability $\frac{j}{k}$ of matching one of the j previous hashes (creating the first collision), and probability $1 - \frac{j}{k}$ of not creating a collision. Hence the recurrence: $q_{j+1} = q_j \left(1 - \frac{j}{k}\right)$. It comes the exact:

$$p_n = 1 - q_n \quad \text{with} \quad q_n = \prod_{j=0}^{n-1} \left(1 - \frac{j}{k}\right) = \frac{k!}{(k-n)!\, k^n}$$

That is also $q_n = P(k,n)/k^n = C(k,n)\, n!/k^n = \binom{k}{n} \frac{n!}{k^n}$, where $P(k,n)$ is the number of permutations of n things among k, and $C(k,n) = \binom{k}{n}$ is the number of

combinations of n things among k (beware that k and n are reversed in this answer compared to the linked references).

This is fine for low k and n, but impractical for large values. With $n \ll k$, applying that for small $\epsilon > 0$ it holds that $\ln(1-\epsilon) \lessapprox -\epsilon$, we get

$$\ln(q_n) = \sum_{j=0}^{n-1} \ln\left(1 - \frac{j}{k}\right) \lessapprox -\frac{1}{k} \sum_{j=0}^{n-1} j = -\frac{n\,(n-1)}{2k}$$

In summary we have exactly

$$p_n = 1 - q_n = 1 - \frac{k!}{(k-n)!\, k^n} \qquad = 1 - P(k,n)/k^n$$

$$= 1 - \binom{k}{n} \frac{n!}{k^n} \qquad = 1 - C(k,n)\, n!/k^n$$

and as fair approximations when the above is inconvenient

$$p_n = 1 - q_n \quad \text{with} \quad \begin{array}{ll} q_n \lessapprox e^{-n\,(n-1)/(2k)} & (\text{assuming } n \ll k) \\ q_n \approx e^{-n^2/(2k)} & (\text{additionally assuming large } n) \end{array}$$

From this we can derive that for large k as used in cryptography, or a b-bit hash (where $k = 2^b$), probability and odds of collision for n random messages are approximately

n	p_n of collision	odds of collision
$\sqrt{k} = 2^{b/2}$	$1 - e^{-1/2} \approx 39.3\%$	$\approx 11/17$

$1.177\sqrt{k} \approx 2^{b/2+0.236\ldots}$	$1/2 = 50\%$	$1/1$
$1.414\sqrt{k} \approx 2^{(b+1)/2}$	$1 - e^{-1} \approx 63.2\%$	$\approx 12/7$
$2\sqrt{k} = 2^{b/2+1}$	$1 - e^{-2} \approx 86.5\%$	$\approx 19/3$

The applied cryptographer is often interested in very low probability of collision p_n, as occurring for $n \ll \sqrt{k}$. The above approximation remains mathematically valid and usable for q_j, but can give inaccurate results for $p_j = 1 - q_j$ because the difference is orders of magnitude smaller than either term in the subtraction. We need another method.

Observe there's a collision when the ith hash equals the jth hash, for any pair (i, j) with $1 \le i < j \le n$; and for each of such $n(n-1)/2$ pairs, there is probability $1/k$ of a collision. Probabilities for such $n(n-1)/2$ events are not quite independent, but for low overall probability of collision this remains a valid approximation, as well as approximating very low probability that any of multiple events occurs to the sum of the probabilities of each event. We get these approximations

$$\begin{aligned} p_n &\lessapprox \frac{n\,(n-1)}{2k} && (\text{assuming } n \ll \sqrt{k}) \\ &\lessapprox \frac{n^2}{2k} && (\text{additionally assuming large } n) \end{aligned}$$

For example, with $n = 2^{100}$ distinct random inputs (hypothetically) hashed to $b = 256$ bits with a perfect (or even passable) hash, probability of collision is about $2^{2\cdot 100-1-256} = 2^{-57}$ (less than 8 chances in a million million millions, which can safely be discounted).

In the question, we have a hash with $b = 256$-bit, that can thus take $k = 2^b = 2^{256}$ output values, and $n = 2^{130} + 1$. Here k is large, $n \ll k$ and $n \not\ll \sqrt{k}$, therefore the appropriate formulas are

$$p_n = 1 - q_n \quad \text{with} \quad q_n \approx e^{-n^2/(2k)} \quad (\text{assuming large } k \text{ and } n \ll k)$$

The $+1$ term in $n = 2^{130} + 1$ is negligible, and we get $-n^2/(2k) \approx -\left(2^{130}\right)^2 / \left(2 \cdot 2^{256}\right) = -2^{2\cdot 130-1-256} = -2^3 = -8$, giving $q_n \approx e^{-8} \approx 0.034\%$ and $p_n \approx 99.966\%$.

Notes: $n \approx 2^{130}$ is so large that it is practically impossible to carry that number of operations, much less hashes. The question's body, and the present answer, thus does not contain anything that could help practically towards *finding collisions for a hash function with a 256-bit output size* (as in the question's original title).
The approximations 99.8% and 99.99% in the question are incorrect in their last digit, whatever the rounding convention.

Share Improve this answer Follow edited Jun 28 at 14:58 answered Aug 27 '16 at 19:01 fgrieu

Follow

fgrieu ♦
117k 10 252 490

Thanks for the response. I'm not sure if you're still incrementally building your solution but here are my questions: 1) How do you use the formula: 1 - $\frac{k!}{(k-n)!k^n}$ to calculate a probability x that a collision will occur? Here let's say x=99%- or is it saying that for (k,n) values the output is the probability of a collision? 2) Is this a correct 'plug and chug' of the formula: 1 - $\frac{2^{256}!}{(2^{256}-2^{130})!(2^{256})^{2^{130}}}$? 3) What specific formula in Stirling's approximation should I use and what is the 'plug n chug' version of it with my example? Thanks! – Max Aug 27 '16 at 19:42

4) Does the sqrt(2^256) tell me the probability of a collision occuring with probability of roughly 50% (in my example)? – Max Aug 27 '16 at 19:47

1 @fgrieu, your approximate formula for p_n is correct. Max, is infeasible to calculate those factorials directly, use the approximation given in the answer and solve for k or x, whichever you want in terms of the other. The recurrence in the answer gives exactly your expression $k!/((k-n)!k^n$. – kodlu Aug 28 '16 at 1:40

1 @fgrieu- this is not homework. I am self-studying (don't know of any profs to ask), so that is why I am trying to understand every sentence in the text I am reading. Really appreciate your responses. You addressed my questions and thus accepted your reply. Thank you! :) – Max Aug 28 '16 at 12:15

1 @Logan: the $\log$ I use is to base e (logarithm to base 10 is rare in crypto, and usually written $\log_{10}$). I changed to $\ln$ to avoid confusion. And it holds $\ln(1-\epsilon) \lessapprox -\epsilon$ when $0 < \epsilon < 1$, e.g. $\ln(0.9) \approx -0.10536$. – fgrieu ♦ Jun 28 at 15:04